

MULTI TP8

Siyuan CHEN

Xiru ZHANG

March 2024

Table des matières

B. Contrôleur DMA	2
Question B1	2
Question B2	2
Question B3	2
Question B4	2
C. Architecture matérielle	3
Question C1	3
Question C2	3
Question C3	3
Question C4	3
D. Code de boot	4
Question D1	4
Question D2	4
Question D3	4
E. Application logicielle de traitement d'image	4
Question E1	4
Question E2	4
Question E3	4
Question E4	5
Question E5	5
Question E6	5
F. Exécution sur architecture multi-processeur	6
Question F1	6
Question F2	6
Question F3	6
Question F4	6

G. Réalisation matérielle du LL/SC	6
Question G1	6
Question G2	7
Question G3	7
Question G4	7

B. Contrôleur DMA

Question B1

block_size : nombre d'octet d'un bloc.

latency : nombre de cycle d'accès au bus pour transférer données.

Question B2

Nombre de bloc = $21 * 128 * 128 * 1 / 512 = 672$.

Question B3

Les registres adressables de IOC sont :

- BLOCK_DEVICE_BUFFER : La base de tampon mémoire
- BLOCK_DEVICE_COUNT : Nombre de bloc à transférer
- BLOCK_DEVICE_LBA : Index du premier bloc de fichier.
- BLOCK_DEVICE_OP : Activer un transfert. (Écriture seule)
- BLOCK_DEVICE_STATUS : L'état du bloc périphérique, représente l'état du périphérique.(Lecture seule)
- BLOCK_DEVICE_IRQ_ENABLE : Activer une interruption après le transfert en cas la valeur non-zéro.
- BLOCK_DEVICE_SIZE : Nombre de bloc adressable, représente le nombre de bloc traité par périphérique.(Lecture seule)
- BLOCK_DEVICE_BLOCK_SIZE : La taille d'un bloc.(Lecture seule)

Question B4

L'état interne de IOC sont :

- BLOCK_DEVICE_IDLE : État inactif. Cela indique que le dispositif n'effectue aucune opération de lecture ou d'écriture et est prêt à recevoir de nouvelles commandes.
- BLOCK_DEVICE_BUSY : État occupé. Cela indique que le dispositif est en train de traiter une opération et ne peut pas accepter de nouvelles commandes pour le moment.
- BLOCK_DEVICE_READ_SUCCESS : Succès de lecture. Cela signifie que le dispositif a terminé une opération de lecture avec succès.
- BLOCK_DEVICE_WRITE_SUCCESS : Succès d'écriture. Cela signifie que le dispositif a terminé une opération d'écriture avec succès.
- BLOCK_DEVICE_READ_ERROR : Erreur de lecture. Cela indique qu'une erreur s'est produite lors d'une opération de lecture.
- BLOCK_DEVICE_WRITE_ERROR : Erreur d'écriture. Cela indique qu'une erreur s'est produite lors d'une opération d'écriture.

C. Architecture matérielle

Question C1

Le PibusBlockDevice peut introduire des latences dues à ses opérations de lecture et d'écriture sur le disque. Le timeout est un mécanisme qui détermine combien de temps le BCU va attendre une réponse d'un composant avant de déclencher une erreur de timeout. Pendant ces transferts, si la latence dépasse la valeur actuelle du timeout, le BCU pourrait interpréter cela comme un échec de communication et générer une erreur. Afin d'éviter de fausses erreurs de timeout, on doit donc ajuster cette valeur pour qu'elle reflète le comportement du PibusBlockDevice.

En général, pour calculer cette valeur, on doit prendre en compte la latence maximale possible du PibusBlockDevice et ajouter une marge pour les retards imprévus ou les accès concurrentiels sur le bus, un timeout raisonnable pourrait être quelques fois ce chiffre, comme 2000 cycles.

Question C2

L'adresse de base du segment associé au IOC : 0x92000000

La longueur du segment associé au IOC : 0x00000020 = 32 octets

Les longueurs des segments associés aux composants ICU, TTY et TIMER :

- La longueur du segment associé au ICU : 32 octets * nombre processeur.
- La longueur du segment associé au TIMER : 16 octets * nombre processeur.
- La longueur du segment associé au TTY : 16 octets * nombre processeur.

Question C3

Nombre de maître : Nombre de processeur + 2(IOC et DMA)

Nombre de cible : 8(ROM, RAM, TTY, FBF, ICU, TIMER, DMA, IOC)

Question C4

Le nombre de de lignes d'interruption en provenance de DMA et IOC : 1

Le nombre de de lignes d'interruption en provenance de TTY et TIMER : nombre de processeurs.

Le nombre de lignes d'interruptions sortantes : nombre de processeurs.

La façon des IRQs provenant des périphériques connectées sur les ports IRQ_IN[i] du composant ICU :

- IRQ_IN[0] : connectée à IRQ DMA.
- IRQ_IN[1] : connectée à IRQ IOC.
- IRQ_IN[pair] : connectée à IRQ TIMER pour chaque processeur.
- IRQ_IN[impair] : connectée à IRQ TTY pour chaque processeur.

D. Code de boot

Question D1

Parce que les piles d'exécution sur les N processeurs doivent être strictement disjointes. Cette méthode permet de s'assurer que ces N processeurs ne modifient pas les données des autres tâches.

Question D2

Le mécanisme général utilisé par le système d'exploitation pour router les différentes lignes d'interruption entrantes sur le composant ICU vers différents processeurs repose sur la table des vecteurs d'interruption. Lorsqu'une interruption est générée par un périphérique, l'ICU reçoit le signal et utilise un codeur de priorité pour déterminer la priorité de cette interruption, il masque temporairement toutes les autres interruptions de priorité inférieure, signale au processeur via le signal ICU_INT, et transmet le vecteur d'interruption, qui est l'index vers la table des vecteurs.

Question D3

- 0b0000001111
- 0b0000110000
- 0b0011000000
- 0b1100000000

E. Application logicielle de traitement d'image

Question E1

Les arguments de `_ioc_read()` :

- `lba` : L'index du premier bloc.
- `buffer` : L'adresse de base du tampon mémoire.
- `count` : Le nombre des blocs à transférer.

Cette fonction attend que le transfert soit terminé pour rendre la main, en appelant la fonction noyau `_ioc_get_lock()` qui est une fonction bloquante.

Question E2

La fonction `_ioc_completed()` sans argument vérifie la complétude d'un transfert E/S, et remet les variables `_ioc_done` et `_ioc_get_lock` à 0.

Question E3

Lorsqu'on saisit une caractère du clavier, il affiche toujours la première image. Si le cache n'est pas correctement mis à jour ou invalidé pour refléter les changements dans la mémoire, les données lues par le processeur depuis le cache peuvent

être obsolètes. Bien que les données des images en mémoire aient été mises à jour, le cache n'ayant pas été synchronisé en conséquence, le processeur continue d'afficher la première image sans passer aux images suivantes.

Question E4

La condition sortie de l'automate SNOOP_FSM de l'état IDLE :

- Écriture externe correspondant à une ligne mise en cache localement (cache hit).
- Écriture externe correspondant à une ligne de cache demandée (wait hit).
- Écriture externe correspondant à une adresse LLSC en attente.

La stratégie adoptée par le SNOOP_FSM lors du traitement des hits externes est l'invalidation. Lorsqu'une opération d'écriture externe est détectée comme pouvant potentiellement entrer en conflit avec les données du cache local, elle demande l'invalidation de la ligne de cache concernée afin de maintenir la cohérence du cache, au lieu de mettre à jour les données dans le cache.

Question E5

La détection de plusieurs hits externes consécutifs pose un problème particulier car elle peut indiquer une contention élevée sur le bus ou une activité intensive de la part d'autres processeurs ou périphériques modifiant des données qui se trouvent également dans le cache local.

Solution au Problème :

Si un nouveau hit externe est détecté (sur une ligne de cache différente de la précédente avant que le premier hit n'ait été complètement traité), le SNOOP FSM demande un flush global du cache (une purge complète). Cette opération vise à invalider toutes les entrées du cache, forçant ainsi le cache à se resynchroniser avec l'état actuel de la mémoire principale.

Question E6

Les durées d'exécutions

	Chargement	Seuillage	Affichage
image 1	41457	436123	107036
image 2	40713	436476	107015
image 3	40769	436476	107015
image 4	40769	436476	107015

TABLE 1 – La durées d'exécution

F. Exécution sur architecture multi-processeur

Question F1

Les phases filtrage et affichage sont effectivement parallélisées.

Question F2

1. Avant de réaliser un transfert, chaque processeur tente d'acquérir un verrou en appelant la fonction système `_ioc_get_lock()`. Cette fonction emploie les instructions atomiques LL (Load Linked) et SC (Store Conditional) pour obtenir de manière atomique le verrou.
2. Si un processeur ne parvient pas à acquérir le verrou (car un autre le détient déjà), il entre dans une boucle d'attente active. Il réessaye périodiquement d'acquérir le verrou jusqu'à ce qu'il y parvienne.
3. Une fois le transfert terminé, le processeur libère le verrou, permettant ainsi aux autres processeurs en attente de tenter d'acquérir le verrou pour réaliser leur propre transfert.

Question F3

La fonction `_ioc_get_lock()` est utilisée pour obtenir un verrou exclusif sur le contrôleur IOC, permettant de sérialiser l'accès en cas de demandes concurrentes provenant de plusieurs processeurs. Le code utilise les instructions atomiques LL (Load Linked) et SC (Store Conditional) pour réaliser cet accès exclusif de manière atomique.

Question F4

La fonction `_ioc_completed()` libère le verrou. Elle vérifie si le transfert est complété en scrutant la variable `_ioc_done`. Une fois le transfert complété et le statut vérifié, la fonction réinitialise les variables de synchronisation `_ioc_done` et `_ioc_lock` à zéro.

La raison pour laquelle il n'est pas nécessaire d'utiliser une instruction atomique spéciale pour libérer le verrou est que la libération du verrou est une opération moins sujette aux conditions de course dans ce contexte spécifique.

G. Réalisation matérielle du LL/SC

Question G1

Cet enregistrement est effectué dans le processeur plutôt que dans la mémoire afin de réduire la surcharge de communication entre le processeur et la mémoire et d'améliorer l'efficacité.

Question G2

Grâce au mécanisme snoop, lorsque processeur P1 effectue une opération d'écriture sur une adresse, les autres processeurs P0 seront informés de l'opération d'écriture et pourront annuler la réservation de la ligne de cache pour cette adresse.

Question G3

L'image est affichée de manière incomplète et dans un ordre incohérent. Car ils sont incapables de percevoir les modifications apportées par chacun aux données partagées et peuvent fonctionner sur la base de données obsolètes.

Question G4

Le mécanisme Snoop est utilisé pour :

- Maintenir la cohérence du cache dans les environnements multiprocesseurs
- Annuler les réservations de lignes de cache pour les opérations LL (Load Linked) lorsque des opérations d'écriture conflictuelles sont détectées.